



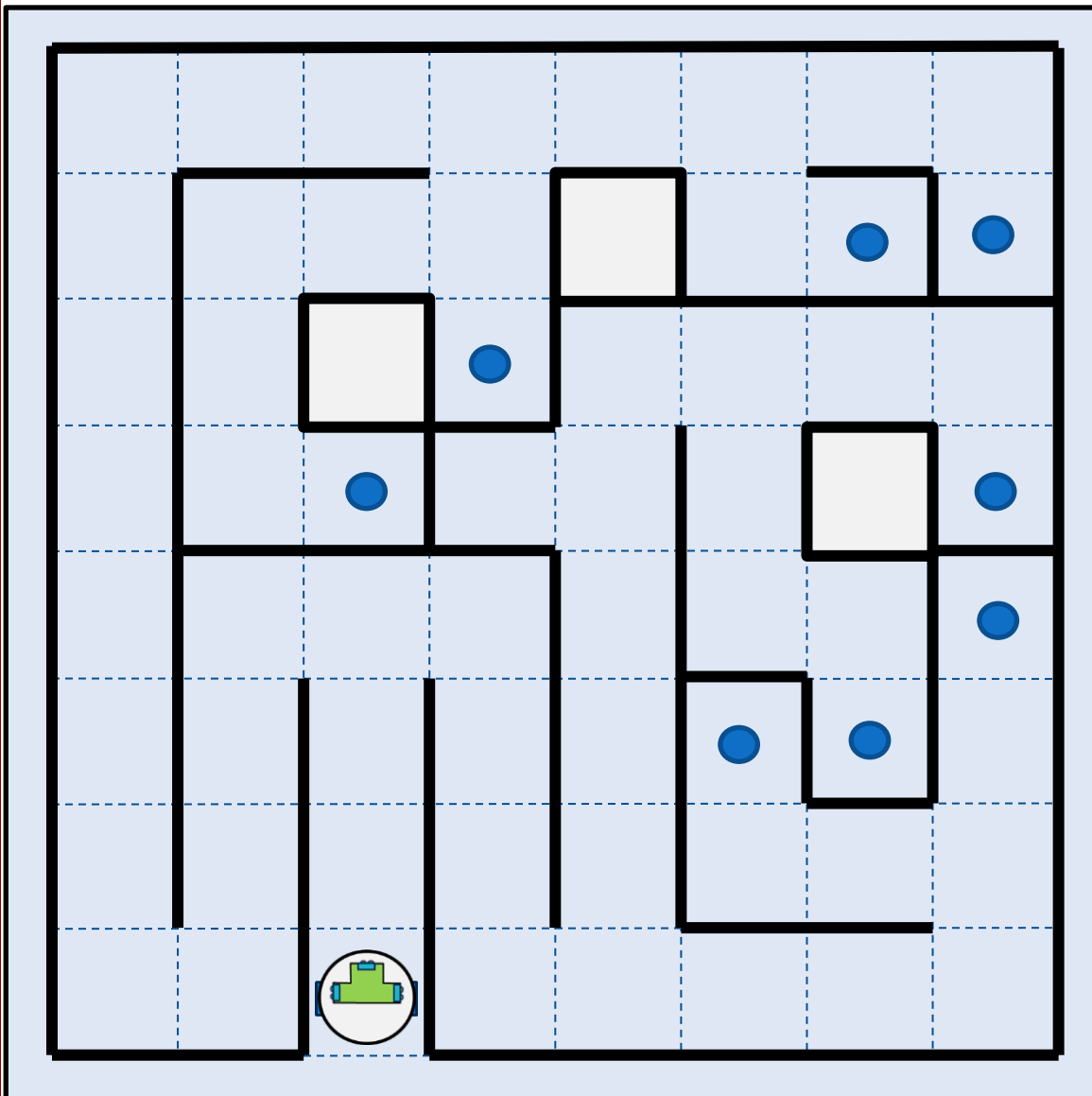
# Exercise 21: Maze Solve & Motor Drive

---

- This is a group exercise – one person submits all files & make sure to include rest of the group in the submission
- Recommended split
  - 2 people on **maze\_solve** implementation & test
  - 2 people on **MtrDrv** implementation & test



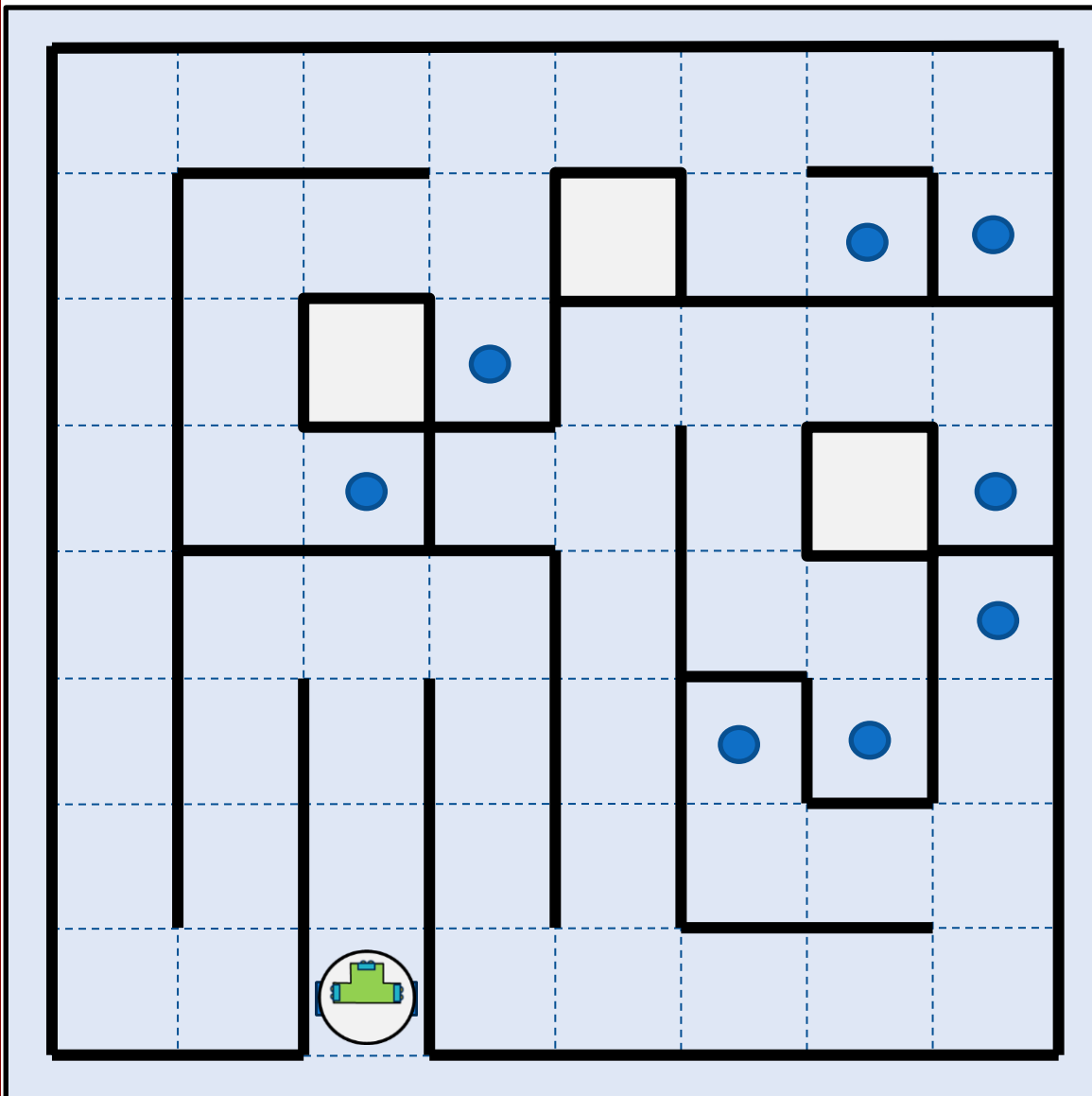
# Exercise 21: Maze Solve



- A magnet could be placed on any of the blue spaces
- The MazeRunner's job is to traverse the maze till it finds it
- Think about the algorithm you would use



# Exercise 21: Maze Solve

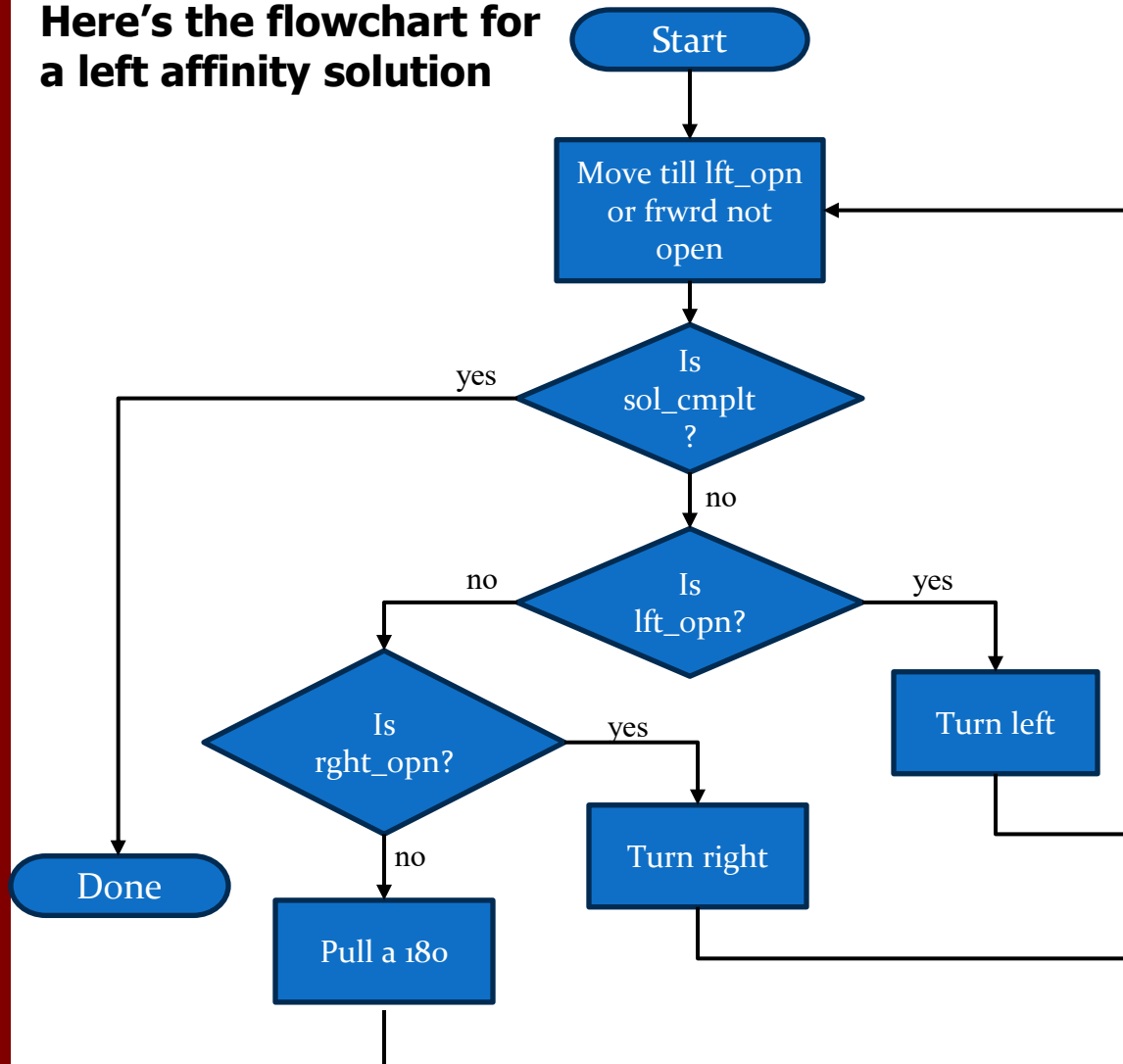


- It can be solved with a left affinity or right affinity algorithm
- Go straight till blocked or preferred opening
- Take preferred opening if possible, otherwise take available direction
- If no available direction, spin 180



# Exercise 21: Maze Solve

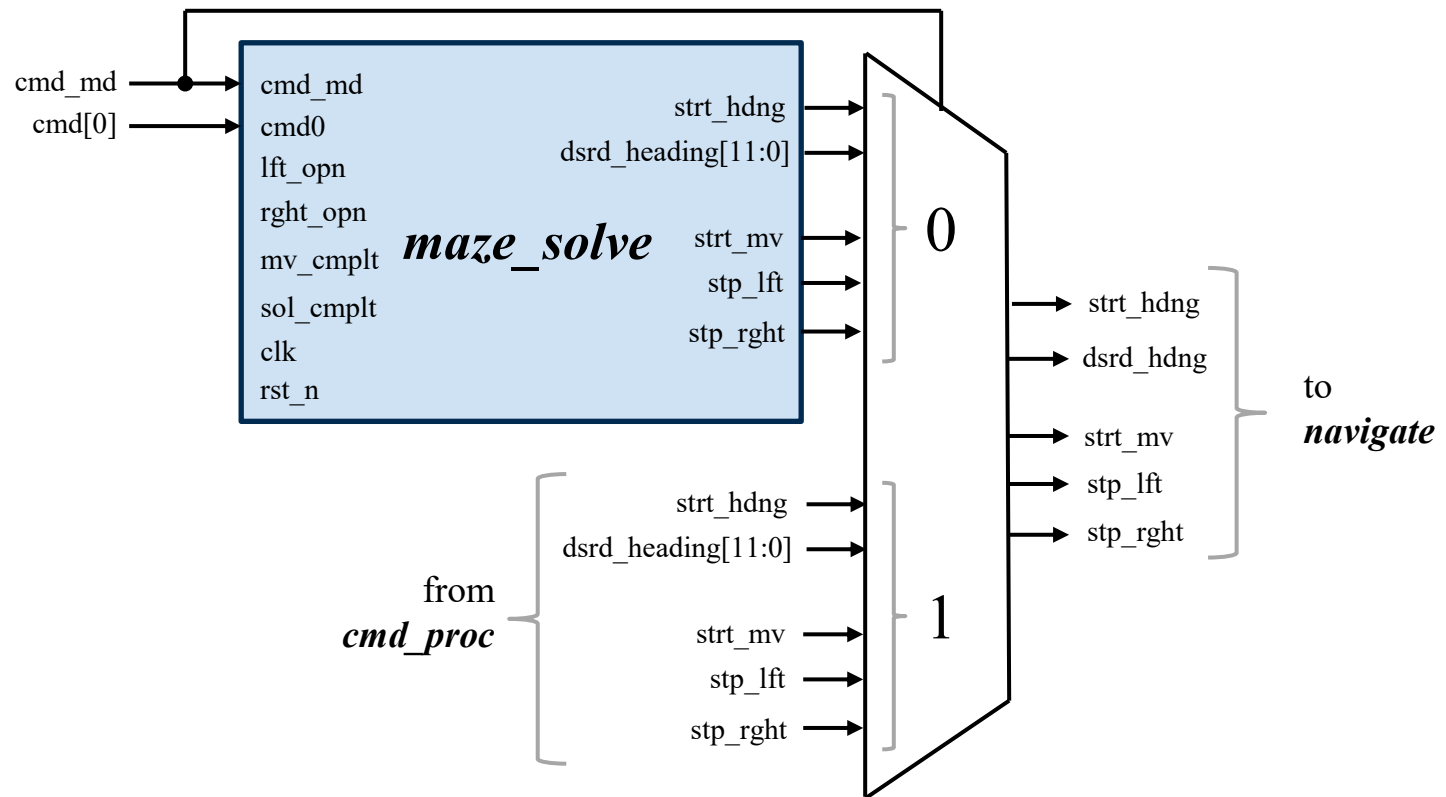
Here's the flowchart for a left affinity solution



- The **maze\_solve** block should be able to perform a left or right affinity solution depending on the state of **cmd[0]**
  - 0: right affinity
  - 1: left affinity
- In addition to the SM, your design will also have to have a **dsrd\_hdng** register to store the newest desired heading
- This register can be reset to 12'h000 (north) because the solve cmd will always be issued right after a gyro calibration



# Exercise 21: Maze Solve



- It controls movement by “hijacking” the controls to the **navigate** block
- Controls that used to come from **cmd\_proc** now originate from **maze\_solve**
- The muxing of these control signals happens at the toplevel of hierarchy. This is shown so you have the context to know the mechanics of how **maze\_solve** can control movement.



# Exercise 21: Maze Solve

- Interface for **maze\_solve**

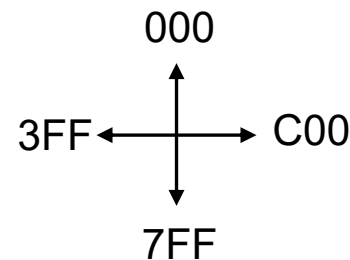
| Signal               | Dir | Description                                                                                                                                                                       |
|----------------------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| clk,rst_n            | in  | 50Mhz clk and asynch active low reset                                                                                                                                             |
| cmd_md               | in  | <b>cmd_md</b> going low is what should kick your SM out of IDLE and have it start doing its thing.                                                                                |
| cmd0                 | in  | LSB of <b>cmd</b> [15:0]. If high perform a left affinity solution.                                                                                                               |
| lft_opn/<br>rght_opn | in  | When <b>mv_cmplt</b> we need to know if lft or right is available direction. Turn toward the direction of affinity if open, otherwise turn opposite if open, otherwise pull a 180 |
| mv_cmplt             | in  | From <b>navigate</b> unit. Asserted when either hdng or mv command is complete                                                                                                    |
| sol_cmplt            | in  | Hey...were done! Comes from hall effect sensor that detects magnet                                                                                                                |
| strt_hdng            | out | Instructs <b>navigate</b> unit to turn mazeRunner toward the new <b>dsrd_hdng</b>                                                                                                 |
| dsrd_hdng            | out | 12-bit heading. Need a register to hold this. Can update 1 clk after <b>strt_hdng</b> asserted.                                                                                   |
| strt_mv              | out | Instructs <b>navigate</b> unit to move forward and stop at either lft/right opening or frwr wall                                                                                  |
| stp_lft/stp<br>_rght | Out | Simply derived from <b>cmd0</b> . Are we performing lft/right affinity?                                                                                                           |



# Exercise 21: Maze Solve

- Your **dsrd\_hdng** register should reset to 12'h000 since a maze solve cmd should always be issued right after a calibration cmd → assumed heading of the **MazeRunner** would be north
- Updates to **dsrd\_hdng** (turning left/right/180) occur through SM control signals
- Keeping the headings on the orthogonal axes listed below can be a bit tricky.
  - Can use the current sign of **dsrd\_hdng**, and whether it was 0 or not

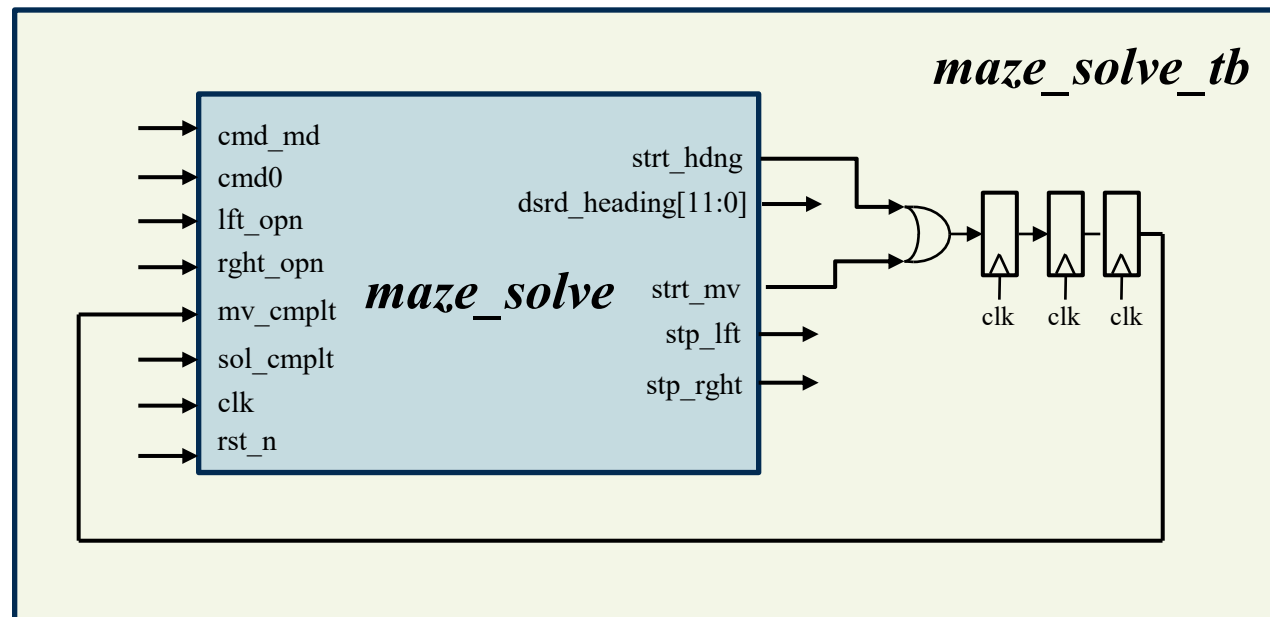
| cmd[11:0] → Heading | Direction |
|---------------------|-----------|
| 12'h000             | North     |
| 12'h3FF             | West      |
| 12'h7FF             | South     |
| 12'hC00             | East      |





# Exercise 21: Maze Solve Testing

- You are going to want to build a unit level testbench to test the basic functionality of **maze\_solve**
- It is slightly painful to test stand alone since it has a significant number of input/outputs, and the correct outputs are not simply a function of inputs, but of the history of moves
- We are now done with all major blocks of the project, and testing this unit more thoroughly in the full chip environment will be easier... but make sure it is at least reasonably wrung out with a unit level testbench

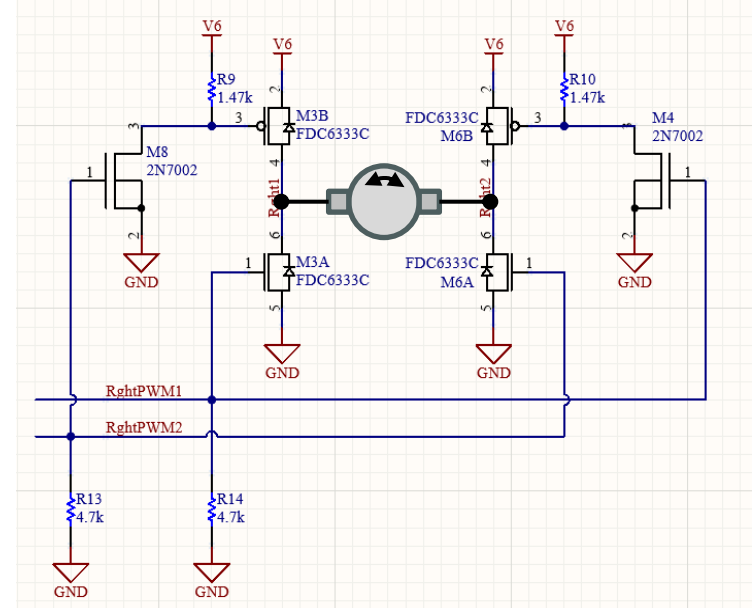






# Exercise 21: Motor Drive

- The motors are driven directly off the battery voltage (nominal 6V)
- With PWM at 50% duty cycle, roughly half the battery voltage is applied across the motor (nominal 3V)
- As batteries wear, their voltage falls
- We need consistent acceleration/speed/braking behavior for proper maze navigation
  - Think about the case of discovering an opening to the left/right. We want to stop in front of that opening so we can turn into it next.
- We need consistent acceleration/speed/braking regardless of battery level → As battery level falls, we want to increase our PWM drive to compensate





# Exercise 21: Motor Drive

- We want to normalize about a battery voltage in the normal range of batteries → Can choose any – we will use 5.4V for MazeRunner
- The desired duty cycle compensation is (for lft, same idea for rht):  
 **$\text{lft\_scaled} = (5.4/\text{batt\_voltage}) * \text{lft\_spd}$**
- The **sensor** block (will be provided later) measures the battery voltage through a voltage divider using the A2D converter
- Accounting for the scaling in the measurement circuit, the upper 8-bits of the battery readings are related to the actual battery as follows:
- The upper 2 bits of **vbatt** are always 11 for the range of interest, so using **vbatt[9:4]** gives us enough range and granularity to perform the PWM duty cycle scaling we desire.

| vbatt[11:4] | Batt Voltage            |
|-------------|-------------------------|
| 0xC0        | 4.74V (too low)         |
| 0xD0        | 5.14V (lowest accepted) |
| 0xDB        | 5.41V (unity scaling)   |
| 0xE0        | 5.534V (attenuated)     |
| 0xF0        | 5.929V                  |
| 0xFF        | 6.3V (fresh battery)    |

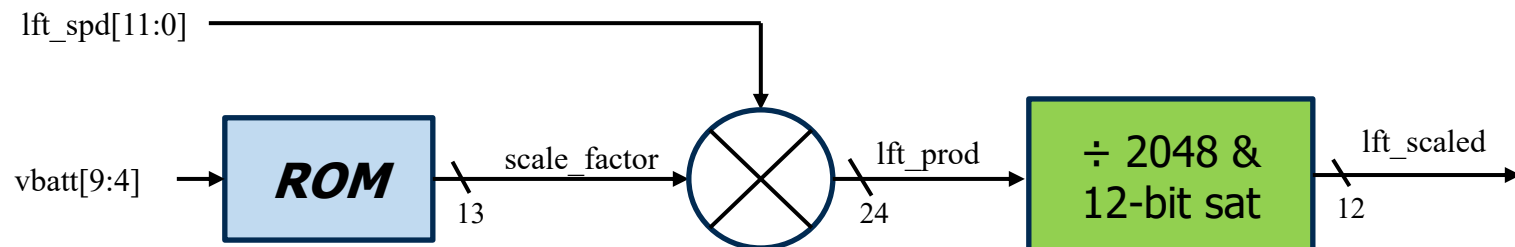


# Exercise 21: Motor Drive

- Division and floating point are high-cost operations  
→ want to do this math in a less expensive way

$$\text{lft\_scaled} = (5.4 / \text{batt\_voltage}) * \text{lft\_spd}$$
$$2048 * \text{lft\_scaled} = (11059 / \text{batt\_voltage}) * \text{lft\_spd}$$

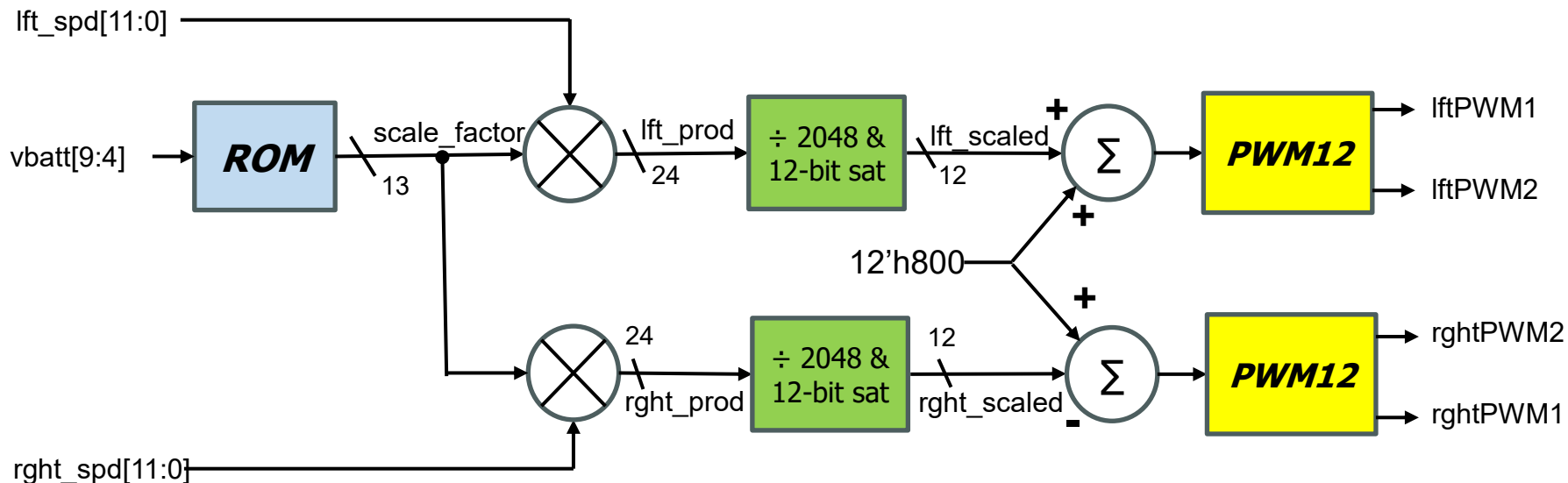
- Instead of doing division, we will do a lookup table of constant coefficients using **vbatt**[9:4] to represent the battery voltage
  - We have plenty of memory on the FPGA, so a 64-entry look up table (ROM) with 13-bit entries is nothing.
  - A model of a ROM is provided on the course page





# Exercise 21: Motor Drive

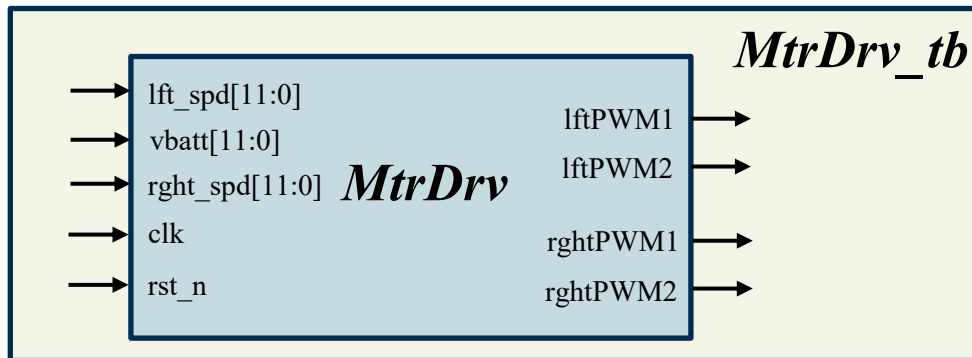
- In addition to scaling, **MtrDrv** also has to convert the 12-bit signed duty cycle to unsigned → this is simply done by adding 0x800
- The motors in MazeRunner are 180° in orientation. We need to invert right motor speed to account for this (i.e., 0x800 - right\_scaled)





# Exercise 21: Motor Drive Testing

- Create a simple testbench



| vbatt[11:4] | Batt Voltage            |
|-------------|-------------------------|
| 0xC0        | 4.74V (too low)         |
| 0xD0        | 5.14V (lowest accepted) |
| 0xDB        | 5.41V (unity scaling)   |
| 0xE0        | 5.534V (attenuated)     |
| 0xF0        | 5.929V                  |
| 0xFF        | 6.3V (fresh battery)    |

- A few recommended test points:
  - Zero in should give 50% duty cycle out regardless of vbatt
  - 0x3FF in with vbatt[11:4]==0xDB should give approx. (slightly less) 75% duty on PWM1 and 25% on PWM2
  - 0x3FF in with vbatt[11:4]==0xD0 should give approx. 76.2% duty on PWM1 and 23% on PWM2
  - 0xC00 in with vbatt[11:4]==0xFF should give approx. 28.5% duty on PWM1 and 71.4% on PWM2
- Make sure you are testing both lft and rght